Assignment #5: Scheme programming

● Graded

Student

Mohammad Abu Shariha

Total Points

50 / 50 pts

Autograder Score

50.0 / 50.0

Passed Tests

Problem 1.1 clamp-bounds - Public (1/1)
Problem 1.1 clamp-bounds - Private (1/1)
Problem 1.2 cleanup-data - Public (1.5/1.5)
Problem 1.2 cleanup-data - Private (1.5/1.5)
Problem 1.3 lesser-decadents - Public (3/3)
Problem 1.3 lesser-decadents - Private (3/3)
Problem 1.4 windowed-average - Public (2/2)
Problem 1.4 windowed-average - Private (2/2)
Problem 2.1 pad-duration - Public (2/2)
Problem 2.1 pad-duration - Private (2/2)
Problem 2.2 fmt-duration - Public (4/4)
Problem 2.2 fmt-duration - Private (4/4)
Problem 2.3 add-durations - Public (4/4)
Problem 2.3 add-durations - Private (4/4)
Problem 3 calculate-subtotal - Public (7.5/7.5)
Problem 3 calculate-subtotal - Private (7.5/7.5)
Imperative Features (0/0)
Forbidden Functions (0/0)

Autograder Results

Problem 1.1 clamp-bounds - Public (1/1)

All tests passed!

Problem 1.1 clamp-bounds - Private (1/1)

Problem 1.2 cleanup-data - Public (1.5/1.5)

All tests passed!

Problem 1.2 cleanup-data - Private (1.5/1.5)

Problem 1.3 lesser-decadents - Public (3/3)

All tests passed!

Problem 1.3 lesser-decadents - Private (3/3)

Problem 1.4 windowed-average - Public (2/2)

All tests passed!

Problem 1.4 windowed-average - Private (2/2)

Problem 2.1 pad-duration - Public (2/2)

All tests passed!

Problem 2.1 pad-duration - Private (2/2)

Problem 2.2 fmt-duration - Public (4/4)

All tests passed!

Problem 2.2 fmt-duration - Private (4/4)

Problem 2.3 add-durations - Public (4/4)

All tests passed!

Problem 2.3 add-durations - Private (4/4)

Problem 3 calculate-subtotal - Public (7.5/7.5)

All tests passed!

Problem 3 calculate-subtotal - Private (7.5/7.5)

Imperative Features (0/0)

Forbidden Functions (0/0)

Submitted Files

code.rkt Download

```
#lang racket/base

(define (clamp-bounds ub lb data)
  (if (null? data)
      '()
      (let ((x (car data)))
        (cons (cond ((> x ub) ub)
                    ((< x lb) lb)
                    (else x))
              (clamp-bounds ub lb (cdr data))))))

(define (cleanup-data data)
  (cond
    ((null? data) '())
    ((boolean? (car data)) (cleanup-data (cdr data)))
    ((string? (car data))
     (cons (string-upcase (car data)) (cleanup-data (cdr data))))
    ((number? (car data))
     (cons (if (even? (car data))
               (/ (car data) 2)
               (- (* (car data) 3) 1))
           (cleanup-data (cdr data))))
    ((list? (car data))
     (append (cleanup-data (car data)) (cleanup-data (cdr data))))
    (else (cleanup-data (cdr data)))))

(define (lesser-decadents data)
  (define (find-lesser val lst)
    (cond
      ((null? lst) '())
      ((< (car lst) val)
       (cons (car lst) (find-lesser val (cdr lst))))
      (else (find-lesser val (cdr lst)))))

  (if (null? data)
      '()
      (cons (list (car data) (find-lesser (car data) (cdr data)))
            (lesser-decadents (cdr data)))))

(define (windowed-average size data)
  (define (sum lst)
    (if (null? lst) 0 (+ (car lst) (sum (cdr lst)))))

  (define (take n lst)
    (if (or (<= n 0) (null? lst))
        '()
        (cons (car lst) (take (- n 1) (cdr lst)))))

  (if (or (null? data) (< size 1))
```

```
    '()
    (let ((window (take size data)))
      (if (< (length window) size) ; Stop if not enough items for a full window
          '()
          (cons (/ (sum window) size)
                (windowed-average size (cdr data)))))))

(define (pad-duration duration)
  (cond
    ((null? (cdr duration)) (list 0 0 (car duration)))
    ((null? (cddr duration)) (list 0 (car duration) (cadr duration)))
    (else duration)))

(define (fmt-duration duration)
  (define (pluralize val unit)
    (string-append (number->string val)
                   " "
                   (if (= val 1) unit (string-append unit "s"))))

  (let* ((padded (pad-duration duration))
         (h (car padded))
         (m (cadr padded))
         (s (caddr padded)))
    (if (and (= h 0) (= m 0) (= s 0))
        "0 Seconds"
        (let ((parts (append (if (> h 0) (list (pluralize h "Hour")) '())
                             (if (> m 0) (list (pluralize m "Minute")) '())
                             (if (> s 0) (list (pluralize s "Second")) '()))))
          (cond
            ((null? (cdr parts)) (car parts))
            ((null? (cddr parts)) (string-append (car parts) " and " (cadr parts)))
            (else (string-append (car parts) ", " (cadr parts) ", and " (caddr parts)))))))))

(define (add-durations leftDuration rightDuration)
  (let* ((left (pad-duration leftDuration))
         (right (pad-duration rightDuration))
         (h1 (car left)) (m1 (cadr left)) (s1 (caddr left))
         (h2 (car right)) (m2 (cadr right)) (s2 (caddr right))

         (s-total (+ s1 s2))
         (m-total (+ m1 m2))
         (h-total (+ h1 h2))

         (s-overflow (quotient s-total 60))
         (s-final (remainder s-total 60))

         (m-with-overflow (+ m-total s-overflow))
         (m-overflow (quotient m-with-overflow 60))
         (m-final (remainder m-with-overflow 60))

         (h-final (+ h-total m-overflow)))

```

```
    (if (> h-final 0)
        (list h-final m-final s-final)
        (if (> m-final 0)
            (list m-final s-final)
            (list s-final)))))

; Moved menu outside the function so it's not redefined every time
(define menu
  '(("vegetable samosa" 4.99)
    ("aloo tikki" 5.35)
    ("paneer pakora" 6.50)
    ("fish curry" 16.99)
    ("tikka masala" 16.35)
    ("chicken saag" 15.00)
    ("daal tadka" 13.10)
    ("malai kofta" 14.35)
    ("mango lassi" 5.00)
    ("spiced tea" 3.50)
    ("coffee" 3.50)))

(define (calculate-subtotal order discounts)

  (define (calculate-order-cost order-list)
    (if (null? order-list)
        0.0
        (let ((first (car order-list)))
          (if (number? first)
              ; Has quantity: (3 "item" ...)
              (let* ((quantity first)
                     (item-name (cadr order-list))
                     (base-price (cadr (assoc item-name menu)))
                     (discount-pair (assoc item-name discounts))
                     (discount-percent (if discount-pair (cadr discount-pair) 0))
                     (cost (* quantity
                              base-price
                              (- 1.0 (/ discount-percent 100.0)))))
                (+ cost (calculate-order-cost (cddr order-list))))

              ; No quantity: ("item" ...)
              (let* ((item-name first)
                     (base-price (cadr (assoc item-name menu)))
                     (discount-pair (assoc item-name discounts))
                     (discount-percent (if discount-pair (cadr discount-pair) 0))
                     (cost (* 1
                              base-price
                              (- 1.0 (/ discount-percent 100.0)))))
                (+ cost (calculate-order-cost (cdr order-list))))))))

  (let ((subtotal (calculate-order-cost order)))
    (/ (round (* subtotal 100)) 100.0)))
```